CMPUT 654: Mathematical Foundations of Modern AI Systems Fall 2026

Lecture 1: Decoder transformers and autoregressive inference

September 1, 2026

Lecturer: Csaba Szepesvári *Note prepared by: Csaba Szepesvári*

Purpose of this note. This is an example of the expected scribe-note style: record what happened in the lecture, define the mathematical objects precisely, and distinguish the main calculation from explanatory remarks. It is an exposition, not a transcript.

Delivery note. Administrative material occupied a substantial part of the meeting. The lecture opened with the harness surrounding a language model and then zoomed in on how a decoder transformer performs inference. Training was described verbally. The lecture also discussed the quadratic cost of attention, key–value caching, maximum context length, and limitations of attention on long contexts. No training objective, loss, or gradient equation was presented. The motivation for learning and the first learning-theoretic results were postponed to Lecture 2.

1.1 The model inside a harness

A deployed AI system is generally larger than one call to a neural network. A *harness* is the surrounding program that constructs the model's context, calls the model, interprets the result, optionally invokes tools, records the new observation, and decides whether to call the model again. It may provide instructions, conversation history, retrieved documents, external memory, or tool outputs. Schematically, one pass through the loop is

$$\boxed{\text{task state}} \xrightarrow{\text{harness}} \boxed{x_{<t}} \xrightarrow{\text{model}} \boxed{p_\theta(\cdot \mid x_{<t})}, \tag{1.1}$$

followed by

$$\boxed{p_\theta(\cdot \mid x_{<t})} \xrightarrow{\text{decoding}} \boxed{\text{token or action}} \xrightarrow{\text{tool/environment}} \boxed{\text{updated task state}}. \tag{1.2}$$

The tool/environment step can be absent, and a harness can use several model calls before returning an answer. The behavior of the complete system can therefore depend on the model parameters, the decoding rule, the maintained state, the tools, and the control loop. The rest of this note zooms into the "model" box in (1.1).

1.2 The inference problem

Given a token prefix $x_{<t} = (x_0, \dots, x_{t-1})$, a language model returns a categorical distribution for the next token:

$$x_{<t} \longmapsto p_\theta(\cdot \mid x_{<t}). \tag{1.3}$$

An inference algorithm selects x_t from this distribution, appends it to the prefix, and repeats. Thus the model and the rule used to decode from it are distinct objects.

Let the vocabulary have size V, and identify token $x_t \in [V]$ with the standard basis vector $e_{x_t} \in \{0,1\}^V$. Include a beginning-of-sequence token x_0. For T prediction positions, define $Z \in \{0,1\}^{T \times V}$ by

$$Z_{t,:} = e_{x_{t-1}}^\top, \qquad t = 1, \dots, T. \tag{1.4}$$

With an embedding matrix $E \in \mathbb{R}^{V\times d}$ and additive absolute-position vectors $P \in \mathbb{R}^{T\times d}$, the input is

$$H^0 = ZE + P, \qquad H^0_{t,:} = E_{x_{t-1},:} + P_{t,:}. \tag{1.5}$$

The one-hot notation makes the map explicit; an implementation performs a row lookup in E.

The complete inference calculation can be summarized by the following block diagram:

$$\boxed{x_{<t}} \longrightarrow \boxed{ZE+P} \longrightarrow \boxed{H^1 \longrightarrow \cdots \longrightarrow H^L} \longrightarrow \boxed{z_t} \longrightarrow \boxed{p_\theta(\,\cdot \mid x_{<t})}. \tag{1.6}$$

The next sections expand the three middle boxes.

Where is position added? Equation (1.5) describes additive absolute positions. They are added once, before the first transformer block; they are not added again in every block. Other position mechanisms enter the computation differently. RoPE applies a position-dependent rotation to queries and keys in each attention layer that uses RoPE. A relative-position bias is likewise added to the attention logits in each layer that uses it. These are alternative architectural choices; the symbol P in (1.5) does not also stand for RoPE.

Remark. The relative-displacement calculation for rotary positions will be developed separately in the homework.

1.3 One fully specified decoder block

Suppose $H^\ell \in \mathbb{R}^{T\times d}$ enters layer ℓ. The lecture described the attention calculation but did not state that a standard transformer uses multiple attention heads. We include that correction here and specify a pre-normalized block with h heads of width d_h.

First define the normalization map. For $u \in \mathbb{R}^d$, let

$$\mu(u) = \frac{1}{d}\sum_{r=1}^{d} u_r, \qquad \sigma^2(u) = \frac{1}{d}\sum_{r=1}^{d}(u_r - \mu(u))^2, \tag{1.7}$$

and, for learned $\gamma_{\ell,m}, \beta_{\ell,m} \in \mathbb{R}^d$ and fixed $\varepsilon > 0$, define

$$\mathrm{LayerNorm}_{\ell,m}(u) = \gamma_{\ell,m} \odot \frac{u - \mu(u)\mathbf{1}}{\sqrt{\sigma^2(u)+\varepsilon}} + \beta_{\ell,m}, \qquad m \in \{1,2\}. \tag{1.8}$$

On a matrix, $\mathrm{LayerNorm}_{\ell,m}$ acts row by row. Layer normalization was not discussed in the lecture; it is included so that the displayed block is fully specified.

At the block-diagram level, a pre-normalized decoder layer performs two residual calculations:

$$H^\ell \longrightarrow \boxed{\text{LayerNorm} \longrightarrow \text{masked multi-head attention}} \xrightarrow{+H^\ell} G, \tag{1.9}$$

$$G \longrightarrow \boxed{\text{LayerNorm} \longrightarrow \text{MLP or MoE}} \xrightarrow{+G} H^{\ell+1}. \tag{1.10}$$

The equations below specify the calculations inside these boxes.

For head $a \in [h]$, let

$$U = \mathrm{LayerNorm}_{\ell,1}(H^\ell), \tag{1.11}$$

$$Q_a = UW^Q_{\ell,a}, \qquad K_a = UW^K_{\ell,a}, \qquad V_a = UW^V_{\ell,a}, \tag{1.12}$$

$$A_a = \mathrm{softmax}_{\mathrm{row}}\left(\frac{Q_aK_a^\top}{\sqrt{d_h}} + M\right), \qquad O_a = A_aV_a, \tag{1.13}$$

where

$$W_{\ell,a}^{Q}, W_{\ell,a}^{K}, W_{\ell,a}^{V} \in \mathbb{R}^{d\times d_h}, \qquad M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases} \tag{1.14}$$

The softmax acts row by row. The causal mask makes the attention weight of every future position exactly zero. Concatenate the head outputs and apply the first residual connection:

$$G = H^{\ell} + [O_1 \mid \cdots \mid O_h]W_{\ell}^{O}, \qquad W_{\ell}^{O} \in \mathbb{R}^{hd_h\times d}. \tag{1.15}$$

The positionwise feed-forward sublayer and second residual connection give

$$R = \mathrm{LayerNorm}_{\ell,2}(G), \tag{1.16}$$

$$H^{\ell+1} = G + \phi\Big(RW_{\ell}^{1} + \mathbf{1}(b_{\ell}^{1})^{\top}\Big)\, W_{\ell}^{2} + \mathbf{1}(b_{\ell}^{2})^{\top}, \tag{1.17}$$

with

$$W_{\ell}^{1} \in \mathbb{R}^{d\times d_{\mathrm{ff}}}, \quad W_{\ell}^{2} \in \mathbb{R}^{d_{\mathrm{ff}}\times d}, \quad b_{\ell}^{1} \in \mathbb{R}^{d_{\mathrm{ff}}}, \quad b_{\ell}^{2} \in \mathbb{R}^{d}. \tag{1.18}$$

For example, $\phi(z) = z\Phi(z)$ coordinatewise is GELU, where Φ is the standard normal cumulative distribution function.

The lecture also mentioned *mixture-of-experts* layers. They replace the single dense feed-forward map by expert maps $f_1, \ldots, f_m$ and a router. For a token representation r, a sparse version has the form

$$f_{\mathrm{MoE}}(r) = \sum_{e\in S(r)} \rho_e(r) f_e(r), \tag{1.19}$$

where $S(r)$ is a small router-selected subset of experts and the router weights $\rho_e(r)$ are normalized over that subset. This replacement changes the positionwise sublayer, not the attention calculation.

Practical models also change normalization, use gated MLPs, reorganize attention heads, and handle positions differently. The equations specify one complete pedagogical block, not the unique modern design.

The key invariant is causal dependence: after L blocks, row $h_t^L = H_{t,:}^L$ depends only on $x_{<t}$.

1.4 From a hidden vector to the next-token distribution

Let $W_U \in \mathbb{R}^{d\times V}$ and $b_U \in \mathbb{R}^{V}$. The final row is converted to logits and then to probabilities:

$$z_t = h_t^L W_U + b_U, \tag{1.20}$$

$$p_\theta(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v})}{\sum_{u=1}^{V}\exp(z_{t,u})}. \tag{1.21}$$

Weight tying sets $W_U = E^{\top}$; it is common but not logically required.

The single-token path is therefore

$$e_{x_{t-1}} \longrightarrow E_{x_{t-1},:} \longrightarrow h_t^L \longrightarrow z_t \longrightarrow p_\theta(\,\cdot \mid x_{<t}). \tag{1.22}$$

The contextual vector h_t^L also depends on the earlier tokens through the masked attention blocks.

1.5 Autoregressive generation

A sampled decoder draws

$$X_t \sim p_\theta(\cdot \mid X_{<t}), \tag{1.23}$$

whereas greedy decoding chooses

$$X_t \in \arg\max_v p_\theta(v \mid X_{<t}). \tag{1.24}$$

In either case the selected token is appended and the procedure repeats until a stopping rule fires.

Greedy decoding is neither sampling from the model nor, in general, global optimization of the probability of the complete sequence. A locally most probable token can lead to poor later continuations.

Practical note. Forget greedy decoding as a default generation rule: it is a bad idea. It throws away nearly all of the predictive distribution at every step, and a locally best token can irreversibly eliminate much better continuations.

Remark. A quantitative counterexample to repeated tokenwise argmax will be developed separately in the homework.

1.6 Attention cost and the key–value cache

For one head on a length-T prefix, both $QK^\top$ and the multiplication of the resulting attention matrix by V require order $T^2 d_h$ arithmetic. A straightforward implementation also materializes a $T \times T$ attention matrix. The causal mask removes the upper triangle, changing a constant but not the quadratic dependence on T.

Autoregressive decoding has additional structure. At step t and in each layer, only the representation of the new token must be advanced. Let its query, key, and value be q_t, k_t, v_t. The attention operation for the new row is

$$\alpha_t = \mathrm{softmax}\left(\frac{q_t K_{\le t}^\top}{\sqrt{d_h}}\right), \qquad o_t = \alpha_t V_{\le t}. \tag{1.25}$$

The earlier keys and values are therefore stored in a *KV cache*. One new step uses order td_h attention arithmetic per head and stores order td_h numbers per head and layer. Summed over a generated sequence of length T, the dense-attention arithmetic is still quadratic.

Why are queries not cached? The query q_i was used to compute the output at position i. A future position $t > i$ forms a new query q_t and compares it with the old key k_i; the resulting weight multiplies the old value v_i. Thus future computation reuses k_i and v_i, but never q_i. Caching old queries would consume memory without avoiding future computation.

1.7 Context length and a bounded-score limitation

A deployed model has a *maximum context length*: an upper bound on the number of prompt and generated tokens available to one model call. Position handling, training range, cache memory,

attention cost, and implementation choices can all contribute to this limit. The ability to accept T tokens does not guarantee that the model uses every one of them effectively.

There is also a simple mathematical pressure against sharply selecting one position from an increasingly long context. For one query attending to T keys, write

$$s_j = \frac{q^\top k_j}{\sqrt{d_h}}, \qquad \alpha_j = \frac{e^{s_j}}{\sum_{r=1}^{T} e^{s_r}}. \tag{1.26}$$

If the scores remain in a fixed bounded range as T grows, no individual softmax weight can remain sharply concentrated. A quantitative upper bound, its proof, and the corresponding entropy consequence will be developed separately in the homework.

The bounded-score premise can fail: a model can make score gaps grow with context length, and multiple heads and layers can implement more complicated selection mechanisms. Thus the observation does not by itself show that transformers cannot use long contexts.

Entropy language. For a probability vector $\alpha \in \mathbb{R}^T$, its entropy is

$$\mathsf{H}(\alpha) = -\sum_{j=1}^{T} \alpha_j \log \alpha_j. \tag{1.27}$$

It is 0 for a deterministic distribution and $\log T$ for the uniform distribution. Thus small entropy describes concentrated attention and large entropy describes diffuse attention.

Related empirical phenomenon. The terms *lost in the middle* and *context rot* refer to observed failures to use long contexts uniformly or reliably, sometimes well below a model's advertised maximum length. These experiments concern complete trained systems and task performance. They are related to the question above, but the bounded-score observation alone does not explain them.

1.8 What was said about training

Training was described only in words. The system is shown many text prefixes together with the tokens that actually followed them. Its predictions are compared with those next tokens, and the shared weights are modified to improve the predictions across the data. Lecture 3 will express this description using probabilistic models, log loss, maximum likelihood, KL divergence, and compression.

1.9 What exactly is contained in the parameter vector?

The symbol θ collects the trainable numerical parameters of the conditional model. For the particular architecture displayed in this note, and assuming that the additive position vectors are learned, one explicit accounting is

$$\theta = \Big(E, P, W_U, b_U, \{W^Q_{\ell,a}, W^K_{\ell,a}, W^V_{\ell,a}\}_{\ell,a}, \{W^O_\ell, W^1_\ell, W^2_\ell, b^1_\ell, b^2_\ell, \gamma_{\ell,1}, \beta_{\ell,1}, \gamma_{\ell,2}, \beta_{\ell,2}\}_\ell\Big). \tag{1.28}$$

With weight tying, $W_U = E^\top$ and is not an independent parameter. Fixed position maps and the causal mask M are not learned parameters. An MoE layer replaces the dense-MLP entries above by the expert and router parameters.

The token prefix, hidden states, queries, keys, values, attention weights, logits, and KV cache are computed quantities, not parts of θ. The decoding rule and the surrounding harness are also outside θ. Thus training the model means choosing the shared numerical parameters in this display; specifying the deployed AI system requires additional choices.

1.10 Take-home messages

- A deployed AI system places the conditional language model inside a harness that can maintain state, invoke tools, and make repeated model calls.
- A decoder transformer maps a prefix to a categorical distribution by embedding tokens, applying causally masked attention and positionwise transformations, and using a softmax readout.
- Absolute position embeddings are added once at the input. RoPE and relative biases instead modify attention in every layer that uses them.
- The learned conditional distribution and the decoding algorithm are different objects. Tokenwise argmax does not solve a general sequence-level optimization problem.
- Dense self-attention has quadratic arithmetic cost in context length. A KV cache avoids recomputing old keys and values during generation; old queries have no future use.
- Maximum context length is an input-capacity limit, not a guarantee of uniform recall or reasoning quality. With bounded attention scores, a single softmax head cannot remain sharply concentrated as the context grows.
- The parameter vector θ contains the learned weights of the conditional model. It does not contain the current activations, KV cache, decoding rule, or harness. This lecture did not yet explain why learning produces useful values of θ.

Notes and references

The descriptions below indicate the role of each reference; they are not a reading list of interchangeable transformer surveys.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. *NeurIPS*, 2017. https://arxiv.org/abs/1706.03762. This paper introduced the transformer architecture. Its complexity table makes the quadratic sequence-length cost of dense self-attention explicit; the architecture in this note is the causal decoder specialization.

[2] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021. https://arxiv.org/abs/2104.09864. This is the source for rotary position embeddings and the relative-displacement identity mentioned in the note.

[3] N. Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. 2019. https://arxiv.org/abs/1911.02150. This paper explains why loading cached keys and values is a central cost of incremental decoding and introduces multi-query attention to reduce that cache and its memory traffic.

[4] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*, 2022. https://arxiv.org/abs/2205.14135. FlashAttention computes exact dense attention by tiling and online softmax, reducing memory traffic and avoiding a materialized quadratic intermediate. It does not remove the quadratic arithmetic of exact dense attention. The course returns to this distinction in Lecture 27.

[5] D. Chiang and P. Cholak. Overcoming a Theoretical Limitation of Self-Attention. *ACL*, 2022. https://aclanthology.org/2022.acl-long.527/. This paper studies a related length-dependent loss of confidence and shows that the obstruction depends on architectural details. Among its remedies is scaling attention logits by $\log T$, the scale needed to keep a bounded-score softmax from becoming increasingly diffuse.

[6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 2024. https://arxiv.org/abs/2307.03172. The experiments vary the position of relevant information in multi-document question answering and key–value retrieval. They show why an advertised context window and robust use of that window are different claims.

[7] K. Hong, A. Troynikov, and J. Huber. Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma technical report, 2025. https://www.trychroma.com/research/context-rot. This report uses "context rot" for empirical degradation as input length grows across several tasks and contemporary models. It is cited as related evidence, not as a consequence of the bounded-score observation.